

Task 3 Microservices with Hazelcast

Автор - Тарас Лусун

Репозиторій: <https://github.com/taraslysun/SA-labs>

Гілка https://github.com/taraslysun/SA-labs/tree/micro_hazelcast

Запускається або через **docker compose up**, або через скрипт **launch_servers.sh.**(лише для linux)

1. Разом з запуском **logging-service** запускається новий екземпляр Hazelcast (або клієнт Hazelcast підключається до своєї ноди кластеру Hazelcast)

Оскільки попередня робота виконувалась з використанням докер-версії hazelcast, то цю лабораторну також вирішив базувати на ній. На початку запуску сервісу створює клієнт використовуючи python-бібліотеку для docker. Оскільки кожен мікросервіс також обгортав в контейнер, то з середовища береться 2 змінні для визначення ip та порту ноди hazelcast. Після цього відповідно створюється мапа зі значеннями.

```
hz_container = start_hz_node(hz_ip, hz_port)
client = hazelcast.HazelcastClient(
    cluster_name='logs-db',
    cluster_members=[f'{hz_ip}:{hz_port}']
)

table = client.get_map('logs-map')
```

Для опрацювання завершення дії сервісу використав декоратор `asynccontextmanager` для асинхронного опрацювання часу роботи програми, а також щоб створити функцію, необхідну для `fastapi`, коли завершується сервіс. Тобто на початку вона зайде в `yield`, і тільки при закінченні роботи

виконає код після finally

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        yield
    finally:
        hz_container.stop()
        hz_container.remove()
        print("Hazelcast container stopped and removed")
        client.shutdown()
```

2. **logging-service** використовує **Hazelcast Distributed Map** для збереження повідомлень замість локальної хеш-таблиці

Див. Пункт 1

3. **facade-service**, з яким взаємодіє клієнт, випадковим чином обирає екземпляр **logging-service** під час відправки POST/GET (GRPC) запитів. Беру список адрес, випадковим чином його перемішую та йду по адресах, поки не отримаю відповідь. Після цього зупиняюсь:

```
def get_service_data(addresses: list) -> dict:
    random.shuffle(addresses)
    for service_addr in addresses:
        try:
            response = requests.get(service_addr)
            return response.json().get("data", {})
        except Exception as e:
            print(f"Exception in {service_addr}: {e}")
    return {"error": "Could not connect to service"}
```

4. якщо обраний екземпляр **logging-service** не доступний, то обирається наступний і т.д.

Див. пункт 3

Завдання

- Запустити три екземпляра **logging-service** (локально їх можна запустити на різних портах), відповідно мають запуститись також три екземпляра **Hazelcast**

Організував все в docker-compose файл для зручності. Оскільки може бути бажання запустити кожен процес окремо – виніс також у скрипт launch_servers.sh. Їхня дія ідентична: створити контейнери -> додати в мережу -> запустити. Запуск на різних портах через docker-compose має приблизно такий вигляд

```
services:
  > Run Service
  facade-service:
    build: ./facade-service
    ports:
      - "8000:8000"
    networks:
      - hazelcast-network

  > Run Service
  logging-service-1:
    build: ./logging-service
    environment:
      - HZ_PORT=5701
      - HZ_IP= #
    ports:
      - "8001:8001"
    networks:
      - hazelcast-network
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock

  > Run Service
  logging-service-2:
    build: ./logging-service
    environment:
      - HZ_PORT=5702
      - HZ_IP= #
    ports:
      - "8002:8001"
    networks:
      - hazelcast-network
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

Використовуються змінні середовища HZ_PORT і HZ_IP для того, щоб запустити контейнер з hazelcast нодою та долучити її до клієнта. В shell-скрипті ip-адреса визначається автоматично. Також для того, щоб контейнер із сервісом міг бачити та спілкуватися з нодою hazelcast, потрібно додати зв'язок через сокет (параметр volume). Після запуску через скрипт

отримаємо наступний список контейнерів:

STATUS	PORTS	NAMES
Up 2 minutes	0.0.0.0:8008->8008/tcp, :::8008->8008/tcp	config-server
Up 2 minutes	0.0.0.0:5703->5701/tcp, [::]:5703->5701/tcp	hazelcast-5703
Up 2 minutes	0.0.0.0:8007->8007/tcp, :::8007->8007/tcp	messages-service
Up 2 minutes	0.0.0.0:5702->5701/tcp, [::]:5702->5701/tcp	hazelcast-5702
Up 2 minutes	0.0.0.0:8003->8001/tcp, [::]:8003->8001/tcp	logging-service-3
Up 2 minutes	0.0.0.0:5701->5701/tcp, :::5701->5701/tcp	hazelcast-5701
Up 2 minutes	0.0.0.0:8002->8001/tcp, [::]:8002->8001/tcp	logging-service-2
Up 2 minutes	0.0.0.0:8001->8001/tcp, :::8001->8001/tcp	logging-service-1
Up 2 minutes	0.0.0.0:8000->8000/tcp, :::8000->8000/tcp	facade-service

- Через HTTP POST записати 10 повідомлень `msg1-msg10` через **facade-service**
- Показати які повідомлення отримав кожен з екземплярів **logging-service** (це має бути видно у логах сервісу)

Запустив простеньку програму з запису:

```
import requests

for i in range(1,11):
    resp = requests.post("http://0.0.0.0:8000", json={"msg":"msg"+str(i)})
    print(resp.text)
```

Результат по різних сервісах:

1)

```
$ docker run --name logging-service-1 --network hazelcast-network -p 8001:8001
-e HZ_PORT=5701 -e HZ_IP=192.168.1.55 -v /var/run/docker.sock:/var/run/docker.sock
logging-service-1
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
Starting Hazelcast container on port 192.168.1.55:5701
Hazelcast container hazelcast-5701 is running on port 5701
INFO: 172.18.0.2:60660 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:60672 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:60678 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:60680 - "POST /logs HTTP/1.1" 201 Created
```

2)

```
$ docker run --name logging-service-2 --network hazelcast-network -p 8002:8001
-e HZ_PORT=5702 -e HZ_IP=192.168.1.55 -v /var/run/docker.sock:/var/run/docker.sock
logging-service-2
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
Starting Hazelcast container on port 192.168.1.55:5702
Hazelcast container hazelcast-5702 is running on port 5702
INFO: 172.18.0.2:56410 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:56418 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:56434 - "POST /logs HTTP/1.1" 201 Created
```

3)

```
$ docker run --name logging-service-3 --network hazelcast-network -p 8003:8001
-e HZ_PORT=5703 -e HZ_IP=192.168.1.55 -v /var/run/docker.sock:/var/run/docker.s
ock logging-service-3
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
Starting Hazelcast container on port 192.168.1.55:5703
Hazelcast container hazelcast-5703 is running on port 5703
INFO: 172.18.0.2:41042 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:41056 - "POST /logs HTTP/1.1" 201 Created
INFO: 172.18.0.2:41058 - "POST /logs HTTP/1.1" 201 Created
```

- Через *HTTP GET* з **facade-service** прочитати повідомлення

Читаємо повідомлення:

```
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
{ "status": "success", "data": { "messages": "message service placeholder", "logs": "msg5,msg6,msg10,msg2,msg4,msg3,msg1,msg9,msg7,msg8" } }
```

Оскільки мапа не посортована, то і повідомлення відповідно також

- *Вимкнути один/два екземпляри logging-service (разом з ним мають вимикатись й ноди Hazelcast) та перевірити чи зможемо прочитати повідомлення*

Вимикаючи екземпляри сервісу бачимо, що ноди hazelcast теж вимикаються

```
INFO:      172.18.0.2:41618 - "POST /logs HTTP/1.1" 201 Created
INFO:      172.18.0.2:33586 - "GET /logs HTTP/1.1" 200 OK
^CINFO:      Shutting down
INFO:      Waiting for application shutdown.
INFO:      Application shutdown complete.
INFO:      Finished server process [1]
Hazelcast container stopped and removed
```

PORTS	NAMES
0.0.0.0:8000->8000/tcp, :::8000->8000/tcp	facade-service
0.0.0.0:5703->5701/tcp, [::]:5703->5701/tcp	hazelcast-5703
0.0.0.0:8003->8001/tcp, [::]:8003->8001/tcp	logging-service-3
0.0.0.0:8008->8008/tcp, :::8008->8008/tcp	config-server
0.0.0.0:8007->8007/tcp, :::8007->8007/tcp	messages-service

Читаємо дані:

```
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
},
{
  "status": "success",
  "data": {
    "messages": "message service placeholder",
    "logs": "msg5,msg6,msg3,msg8,msg1,msg9,msg7,msg10,msg2,msg4"
  }
}
```

Результат успішний, повідомлення розташовані в іншому порядку, ніж попередні, тому можемо зробити висновок, що все працює.

Також бачимо, як опрацьовуються дані з серверів в метод GET, якщо відповіді немає -> взяти результат в іншій ноді

```
Exception in http://logging-service-1:8001/logs: HTTPConnectionPool(host='logging-service-1', port=8001): Max retries exceeded with url: /logs (Caused by NameResolutionError("<urllib3.connection.HTTPConnection object at 0x7799768ef2c0>: Failed to resolve 'logging-service-1' ([Errno -3] Temporary failure in name resolution)"))
Exception in http://logging-service-2:8001/logs: HTTPConnectionPool(host='logging-service-2', port=8001): Max retries exceeded with url: /logs (Caused by NameResolutionError("<urllib3.connection.HTTPConnection object at 0x7799768ef470>: Failed to resolve 'logging-service-2' ([Errno -3] Temporary failure in name resolution)"))
INFO:      172.18.0.1:39720 - "GET / HTTP/1.1" 200 OK
Exception in http://logging-service-2:8001/logs: HTTPConnectionPool(host='logging-service-2', port=8001): Max retries exceeded with url: /logs (Caused by NameResolutionError("<urllib3.connection.HTTPConnection object at 0x779976930710>: Failed to resolve 'logging-service-2' ([Errno -3] Temporary failure in name resolution)"))
INFO:      172.18.0.1:39728 - "GET / HTTP/1.1" 200 OK
INFO:      172.18.0.1:39732 - "GET / HTTP/1.1" 200 OK
INFO:      172.18.0.1:39736 - "GET / HTTP/1.1" 200 OK
INFO:      172.18.0.1:39748 - "GET / HTTP/1.1" 200 OK
```

Додаткове завдання

Також виконав додаткове завдання на створення сервісу для контролю адрес:

```
SERVICES = {
    "logging-service" : (os.getenv("LOGGING_ADDRESSES") or "").split(';'),
    "messages-service" : (os.getenv("MESSAGES_ADDRESSES") or "").split(';')
}

@app.get("/services", status_code=200)
def get_service(service_name: ServiceName) -> dict:
    if service_name.name not in SERVICES:
        return {
            "status": "error",
            "data": "Service not found"
        }

    return {
        "status": "success",
        "data": SERVICES[service_name.name]
    }
```

Він в пам'яті тримає мапу значень сервісів, які при запуску завантажує з середовища. Має приблизно такий вигляд:

```
LOGGING_ADDRESSES=http://logging-service-1:8001/logs;http://logging-service-2:8001/logs;http://logging-service-3:8001/logs
MESSAGES_ADDRESSES=http://messages-service:8007/messages
```

Facade-service бере з цього сервісу дані та в подальшому їх опрацьовує для того, щоб випадковим чином брати собі сервіси

```
def get_service_addresses(service_name: str) -> list:
    try:
        response = requests.get(CONFIG_ADDRESS, json={"name": service_name})
        return response.json().get("data", [])
    except Exception as e:
        print(f"Exception in {service_name}: {e}")
        return []
```